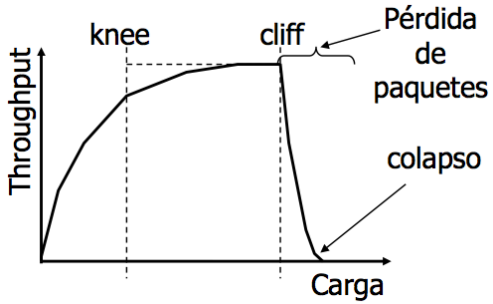
86.12 Comunicación de Datos
TB067 Redes de Comunicaciones
Transporte – TCP Performance

Luis Marrone

fiuba

25 de septiembre de 2024



CC BY

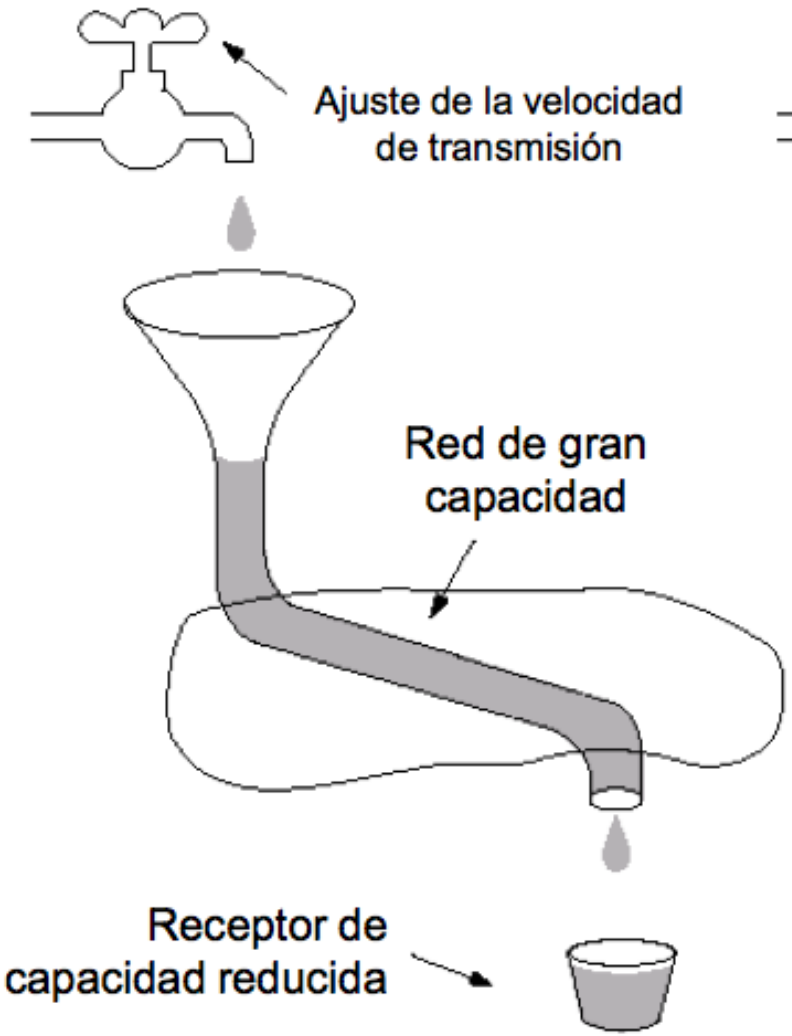
Contenidos

1. Control de Congestión
   - Slow Start – Congestion Avoidance
   - Fast retransmit – Fast Recovery
   - TCP - Cubic
2. Opciones TCP
3. TCP Selective Acknowledge
4. Performance TCP
5. Explicit Congestion Notification en TCP/IP
6. Active Queue Management
7. Nuevo Transporte

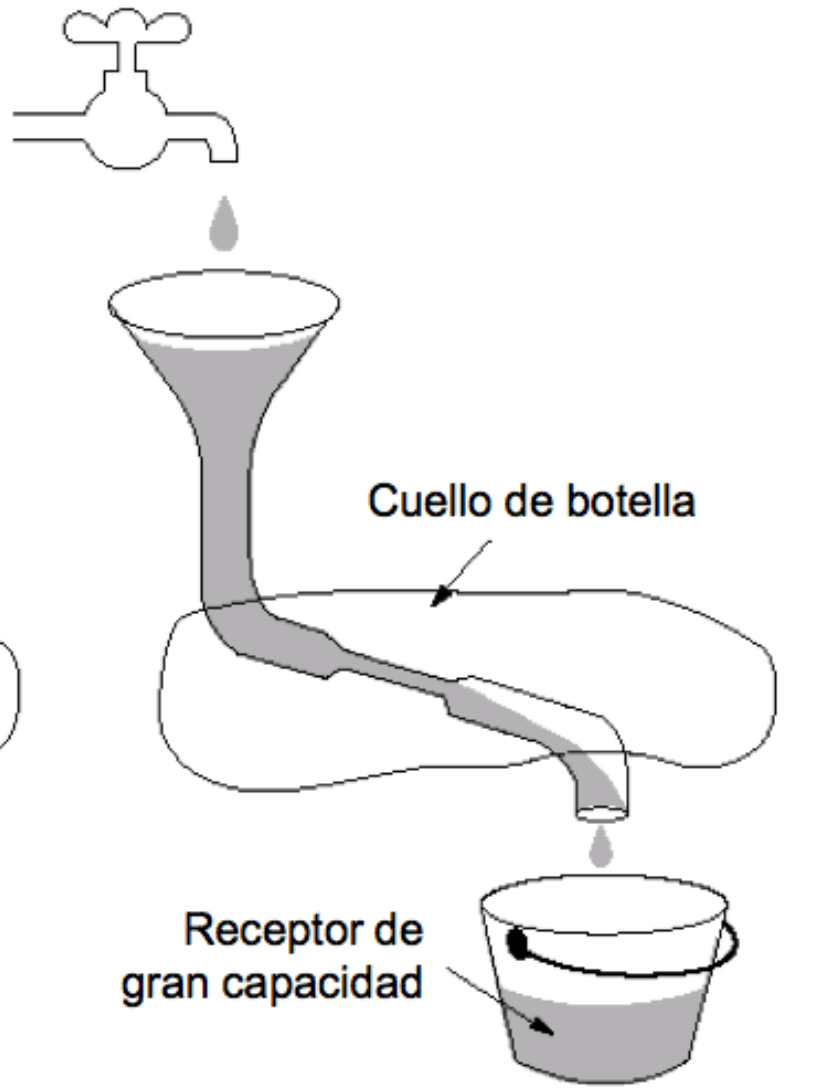
Estamos en:

1. Control de Congestión
   - Slow Start – Congestion Avoidance
   - Fast retransmit – Fast Recovery
   - TCP - Cubic
2. Opciones TCP
3. TCP Selective Acknowledge
4. Performance TCP
5. Explicit Congestion Notification en TCP/IP
6. Active Queue Management
7. Nuevo Transporte

Control de Flujo vs. Control de Congestión

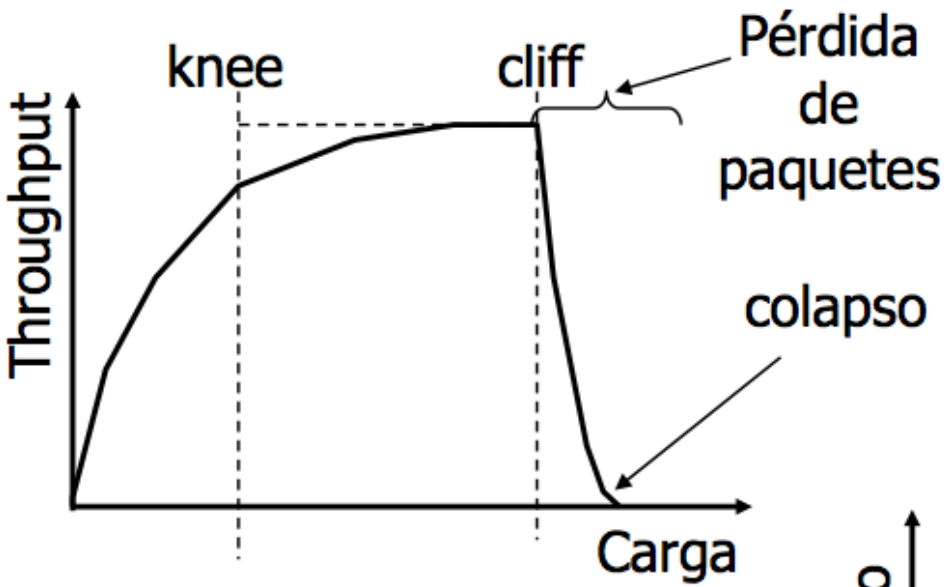


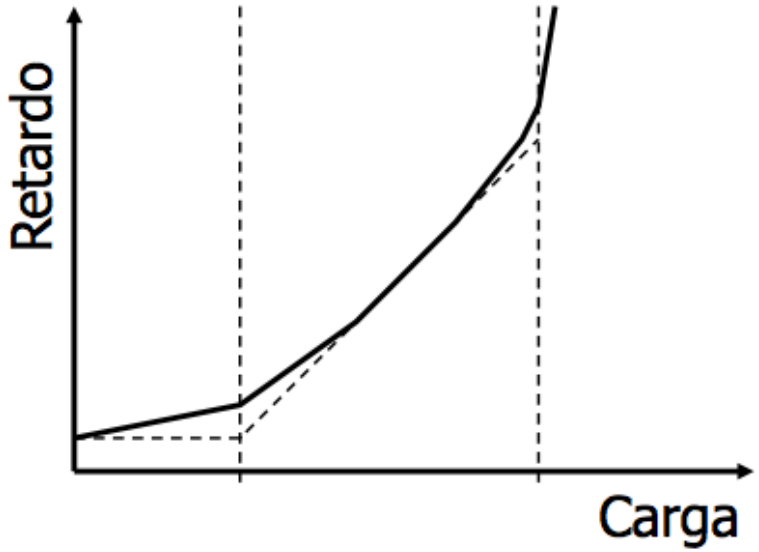
Control de flujo



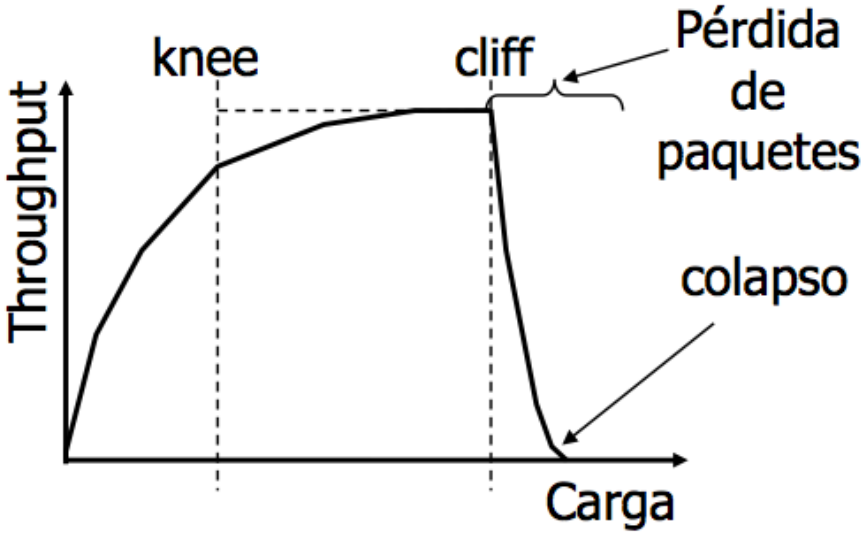
Control de congestión

Throughput vs. Congestión





Controlar vs. Evitar



- Controlar
  - Permanecer a la izquierda del “cliff”
- Evitar
  - Permanecer a la izquierda del “knee”

Modelos

- Modelo end-to-end
  - Los extremos son la fuente de la demanda
  - Los extremos deben estimar los tiempos y grado de congestión y reducir la demanda
  - Los nodos intermedios deben monitorear el estado de la red.
- Modelo basado en la red
  - Los extremos no son confiables
  - El nodo de la red tiene control sobre el tráfico
  - Acciones más rápidas

TCP – Modelo end-to-end

- Se basa en cuatro algoritmos:
  - slow start - congestion avoidance - fast retransmit - fast recovery
- Asocia pérdida de segmento con congestión
- Utiliza tres variables:
  - cwnd: ventana de congestión
  - rcv_win: ventana del receptor. Publicada en el segmento.
  - ssthresh: valor del umbral de “slow start”.
    Determina si slow start o congestion avoidance
- Para el envío
  - ventana = min(rcv_win,cwnd)

Estamos en:

1. Control de Congestión
   - Slow Start – Congestion Avoidance
   - Fast retransmit – Fast Recovery
   - TCP - Cubic
2. Opciones TCP
3. TCP Selective Acknowledge
4. Performance TCP
5. Explicit Congestion Notification en TCP/IP
6. Active Queue Management
7. Nuevo Transporte

Slow Start

- ✓ Se utiliza al comienzo de la sesión o después de la recuperación de una pérdida por retransmisión
- ✓ Inicializa el sistema y descubre la congestión rápidamente.
- ✓ Incrementa cwnd hasta la congestión o hasta *cwnd* $>=$ *ssthreshold*
- ✓ Estima el óptimo cwnd
- ✓ Detecta congestión por pérdida de segmentos
- ✓ Desventajas
  - Detección tardía
  - Enlaces de alta velocidad $--> $ ventanas mayores $--> $ mayor pérdida

Slow Start ...

- ✓ Se inicia con cwnd= IW(Initial Window)

```
If SMSS>2190 bytes:IW=2*SMSS bytes

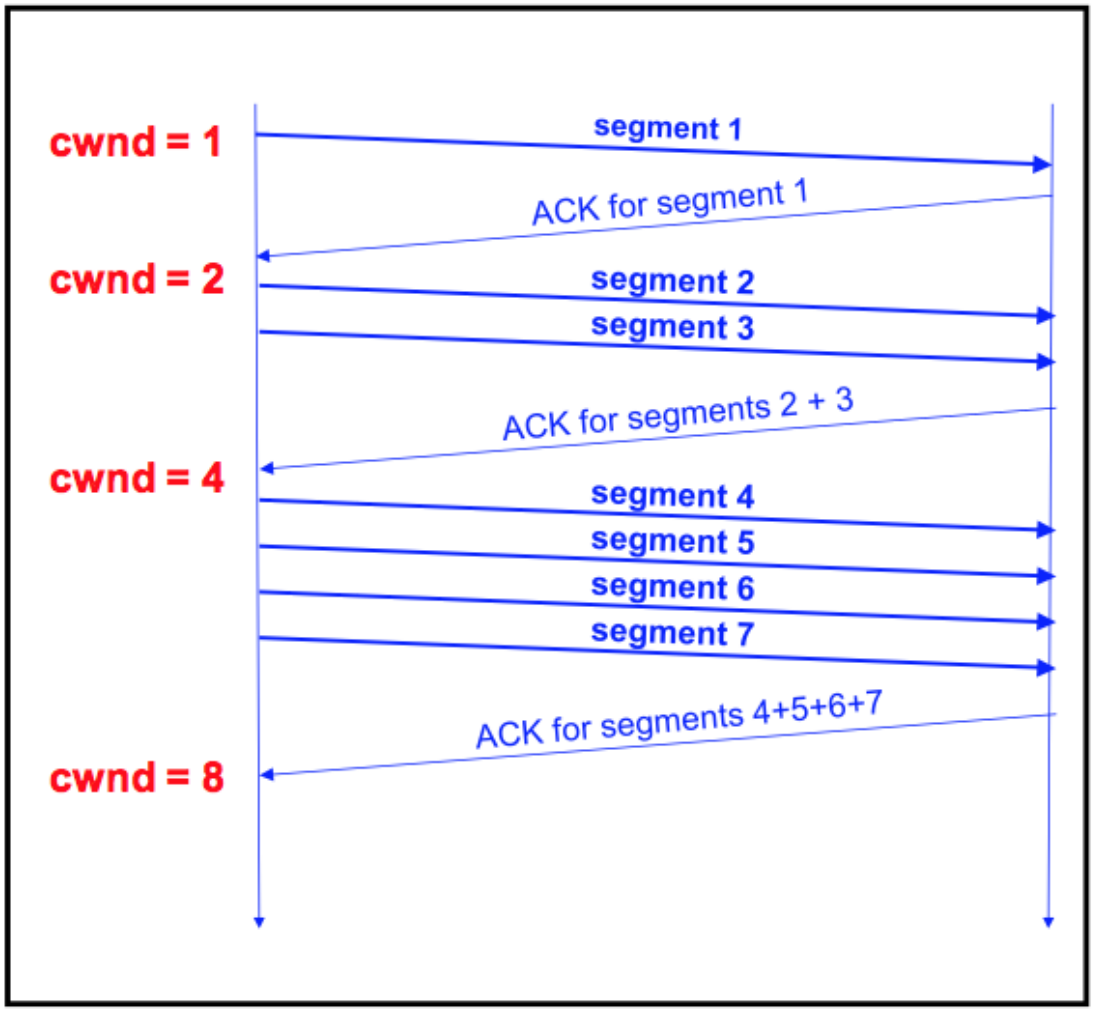
if (SMSS>1095 bytes) & (SMSS <= 2190 bytes):IW=3*SMSS bytes

if SMSS<=1095 bytes:IW=4*SMSS bytes
```

  Interesados en IW $--> $ RFC 3390

- ✓ Después de cada segmento validado
  `cwnd += SMSS`(MSS del emisor)
  Se recomienda:
  `cwnd += min (N,SMSS)` (N=bytes restantes de confirmar)
- ✓ TCP detiene el crecimiento de cwnd cuando `cwnd>=ssthresh`
- ✓ Pese al incremento unitario resulta exponencial

Slow Start – Ejemplo



Congestion Avoidance

$cwnd \geq ssthreshold$

- ✓ $cwnd+ = 1MSS$ por cada RTT
  Se recomienda `cwnd += min (N,SMSS)`
- ✓ Continúa hasta la congestión
- ✓ Alternativa:
  `cwnd += SMSS*SMSS/cwnd`

Congestion Avoidance

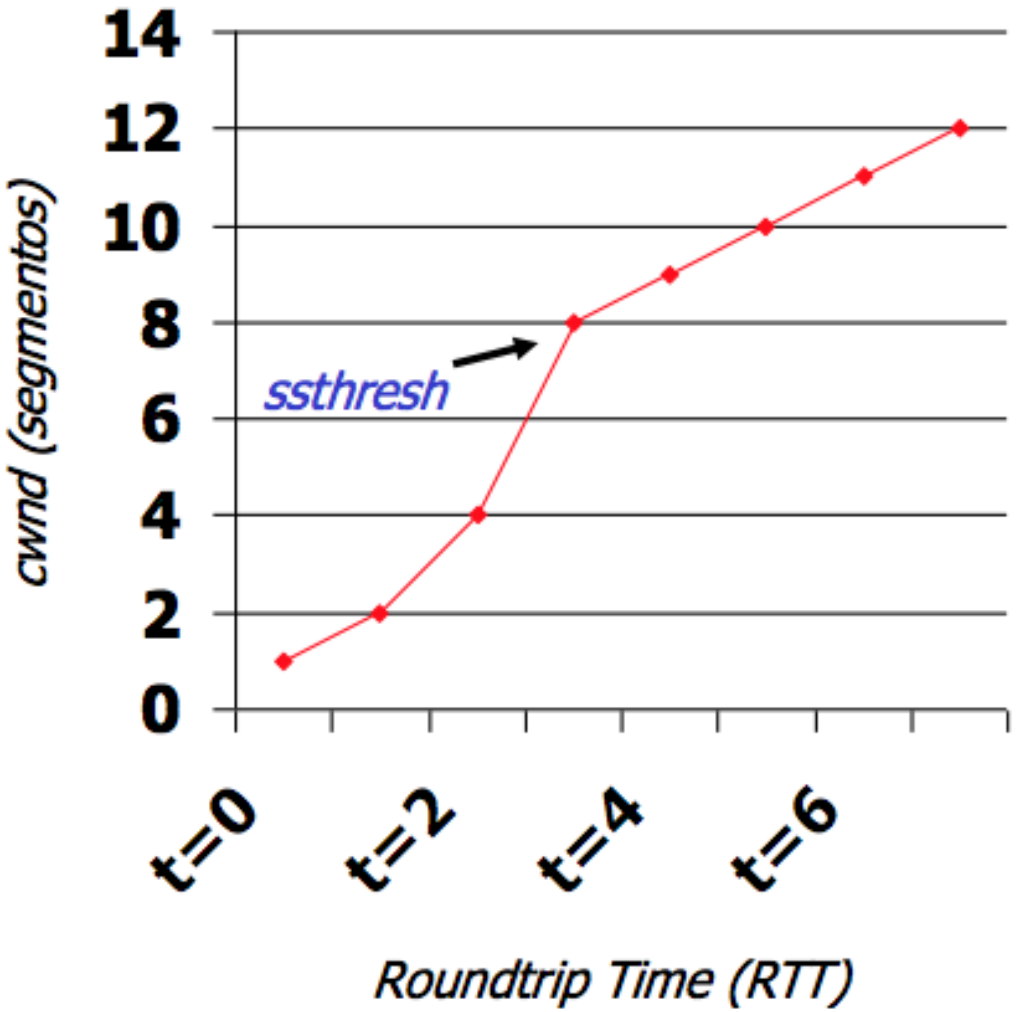
- ✓ Pérdida por RTO pero no retransmitió:
  `ssthresh = max(FlightSize/2, 2*SMSS)`
  FlightSize son los bytes pendientes en la red.

- ✓ Si retransmitió `ssthresh = cte`

- ✓ Luego Slow Start

Combinando...

ssthresh = 8



Armando el Rompecabezas

Inicialmente

$cwnd = 1$

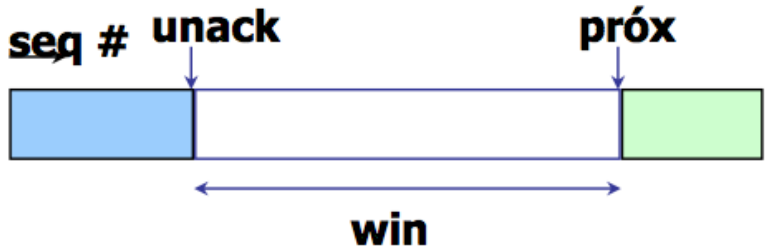
$ssthresh = \infty$

ack recibido
si (cwnd $<$ ssthresh)
/* Slow Start*/
cwnd = cwnd + 1
Tiemout
/* Multiplicative Decrease */

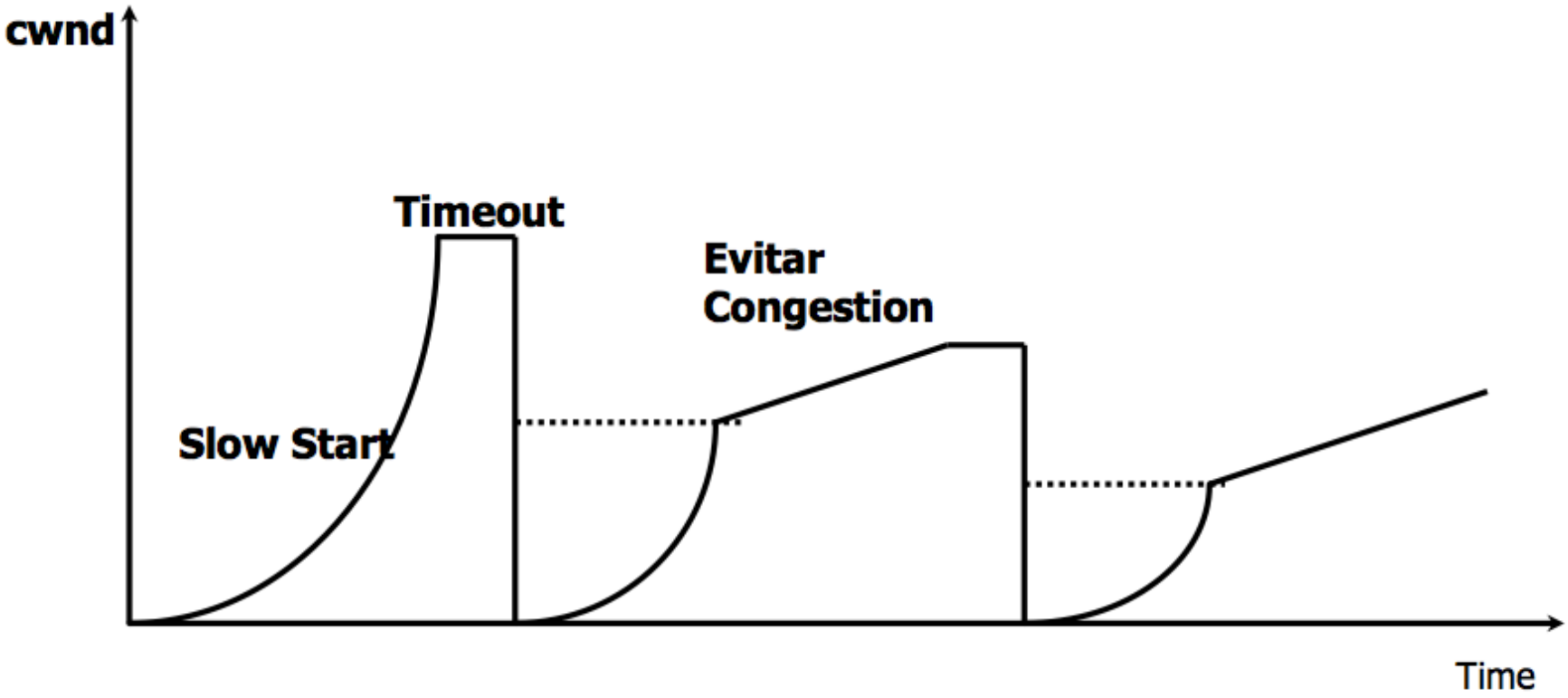
$ssthresh = win/2$

$cwnd = 1$

mientras (próx $<$ unack + win)
transmitir próx segmento
según win = min(cwnd,rcv_win)



Finalmente...



Estamos en:

1. Control de Congestión
   - Slow Start – Congestion Avoidance
   - Fast retransmit – Fast Recovery
   - TCP - Cubic
2. Opciones TCP
3. TCP Selective Acknowledge
4. Performance TCP
5. Explicit Congestion Notification en TCP/IP
6. Active Queue Management
7. Nuevo Transporte

Detección de paquetes perdidos

- ✓ Esperar RTO
- ✓ RTO es usualmente dos veces RTT
- ✓ Degradación de performance
- ✓ No esperar RTO
  - Utilizar mecanismos alternativos
  - Utilizar RTO si no actúan los anteriores

Fast Retransmit

- ✓ Frente a un segmento fuera de orden se envía un ACK.
- ✓ Provoca duplicación de ACKs.
- ✓ Esta duplicación se ve como debida a:
  Paquetes perdidos – Reordenamiento de paquetes
- ✓ No se puede discriminar
- ✓ Si se reciben 3 ACKs duplicados se considera que se debe a un paquete perdido.
- ✓ Al recibir el tercer ACK repetido se retransmite sin esperar el RTO.

Fast Recovery

- ✓ Luego `cwnd = ssthresh + 3 * MSS`
- ✓ Por cada ACK posterior repetido:
  `cwnd += MSS`
- ✓ Hay mútiples variantes

Integrando...

- ✓ Slow Start.
- ✓ Congestion Avoidance.
- ✓ Si aparecen ACKs duplicados
  Fast Retransmit y Fast Recovery.
  Congestion Avoidance.
- ✓ Si RTO
  Slow Start
- ✓ Resumiendo, TCP Reno.
- ✓ Standard RFC 5681 -Septiembre 2009.

Estamos en:

1. Control de Congestión
   - Slow Start – Congestion Avoidance
   - Fast retransmit – Fast Recovery
   - TCP - Cubic
2. Opciones TCP
3. TCP Selective Acknowledge
4. Performance TCP
5. Explicit Congestion Notification en TCP/IP
6. Active Queue Management
7. Nuevo Transporte

Cubic

- ✓ Surje como mejora de performance en redes de alto BDP (Bandwidth-Delay-Product)
- ✓ Evolución de BIC (Binary Increase Congestion Control) (INFCOM 2004)
- ✓ `cwnd` con función cúbica en lugar de lineal
- ✓ Default en Linux
- ✓ RFC 9438 CUBIC for Fast and Long-Distance Networks - Agosto 2023

Principios de diseño

1. Para una mejor utilización y estabilidad de la red, CUBIC utiliza los perfiles cóncavo y convexo de una función cúbica para aumentar el tamaño de la ventana de congestión, en lugar de usar solo un función convexa.

2. Compatible con TCP en redes con RTT cortos y ancho de banda pequeño donde TCP estándar funciona bien.

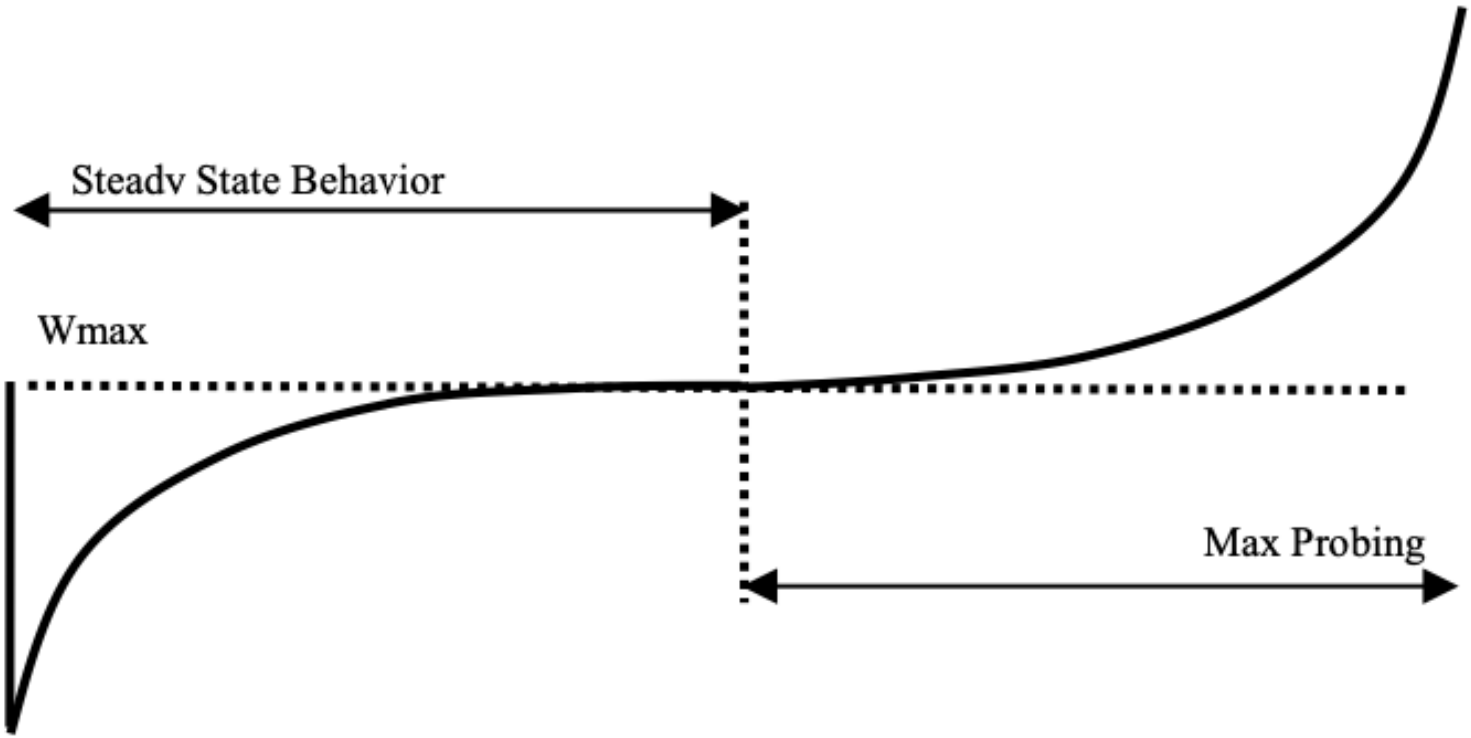
3. Lograr compartir ancho de banda entre flujos con diferentes RTT.

4. Determina el factor multiplicativo de `cwnd` con el fin de equilibrar entre la escalabilidad y velocidad de convergencia.

Funcionamiento

- ✓ Utiliza el tramo cóncavo y el convexo de una función cúbica para el crecimiento de `cwnd`
- ✓ Detectada la congestión registra `cwnd` como W_max y realiza una disminución multiplicativa de `cwnd`
- ✓ Después de ingresar en Congestion Avoidance comienza a aumentar `cwnd` utilizando el perfil cóncavo de la función cúbica que tiene su meseta en W_max, hasta que `cwnd` se convierte en W_max.
- ✓ Luego sigue aumentando con un perfil convexo.

Función CUBIC



Incremento de CUBIC

```
W_cubic(t) = C*(t-K)^3 + W_max

C = constante de "agresividad"

t= tiempo transcurrido desde el comienzo de
Congestion Avoidance

K = cubic_root(W_max*(1-beta_cubic)/C)

beta_cubic es el factor de decrecimiento
```

Modos

- ✓ 1. TCP-friendly region, asegura el mismo throughput que el standard.
- ✓ 2. Región cóncava, `cwnd` es menor que W_max.
- ✓ 3. Región convexa, `cwnd` es mayor que W_max.

Congestión en CUBIC

- ✓ Detección por ACK repetidos o ECN:

```
W_max = cwnd; // save window size before reduction
ssthresh = cwnd * beta_cubic; // new slow-start threshold
ssthresh = max(ssthresh, 2); // threshold is at least 2 MSS
cwnd = cwnd * beta_cubic; // window reduction
beta_cubic = 0,7
```

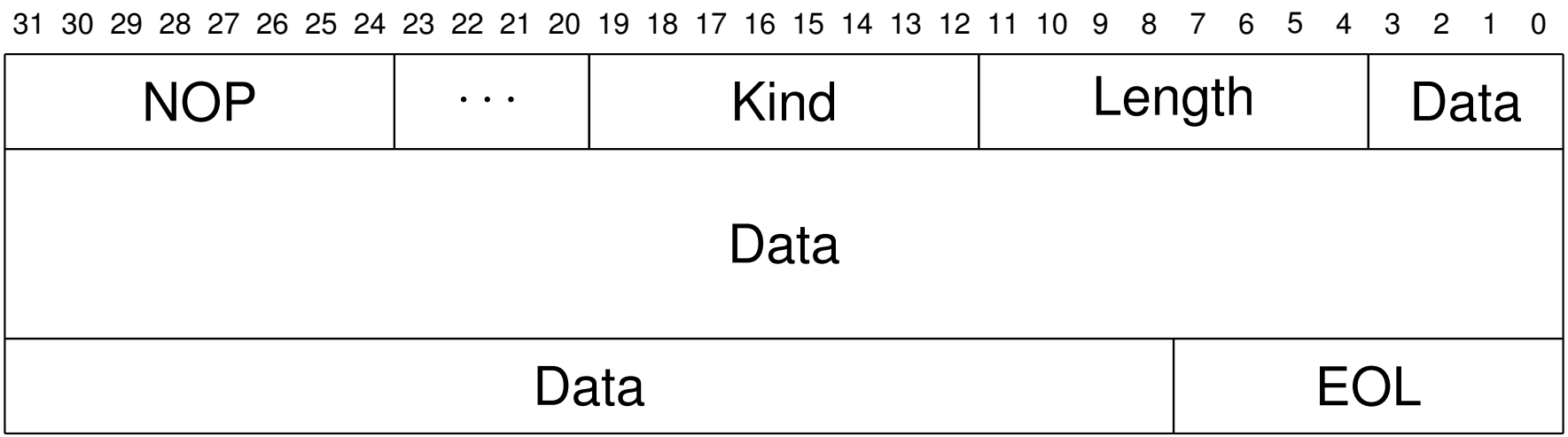
- ✓ Por RTO timeout:
  Standard TCP pero

```
ssthresh = cwnd * beta_cubic; // new slow-start threshold
ssthresh = max(ssthresh, 2); // threshold is at least 2 MSS
```

Estamos en:

1. Control de Congestión
   - Slow Start – Congestion Avoidance
   - Fast retransmit – Fast Recovery
   - TCP - Cubic
2. Opciones TCP
3. TCP Selective Acknowledge
4. Performance TCP
5. Explicit Congestion Notification en TCP/IP
6. Active Queue Management
7. Nuevo Transporte

Estructura Opciones TCP



Opciones - 2024-04-20

Table 13-1 The TCP option values. Up to 40 bytes are available to hold options.

Kind	Length	Name	Reference	Description and Purpose
0	1	EOL	[RFC0793]	End of Option List
1	1	NOP	[RFC0793]	No Operation (used for padding)
2	4	MSS	[RFC0793]	Maximum Segment Size
3	3	WSOPT	[RFC1323]	Window Scaling Factor (left-shift amount on window)
4	2	SACK-Permitted	[RFC2018]	Sender supports SACK options
5	Var.	SACK	[RFC2018]	SACK block (out-of-order data received)
8	10	TSOPT	[RFC1323]	Timestamps option
28	4	UTO	[RFC5482]	User Timeout (abort after idle time)
29	Var.	TCP-AO	[RFC5925]	Authentication option (using various algorithms)
253	Var.	Experimental	[RFC4727]	Reserved for experimental use
254	Var.	Experimental	[RFC4727]	Reserved for experimental use

.-Extraído de TCP/IP Illustrated 2nd. edition

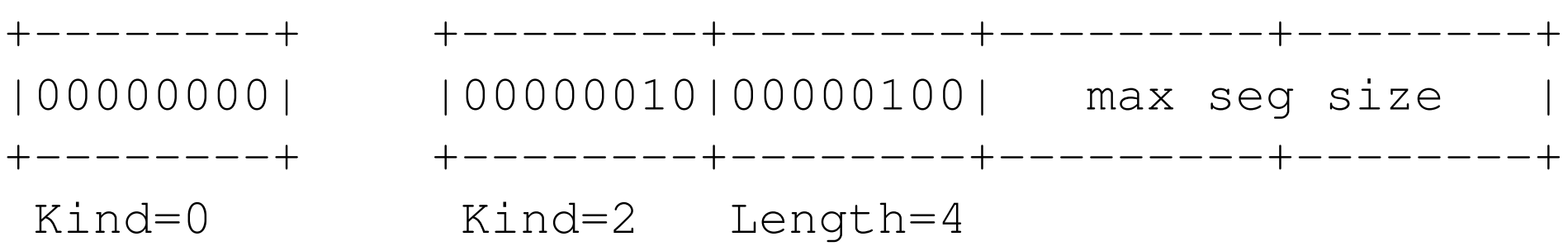
.-RFC 1323 actualizada por RFC 7323

.-RFC 0793 actualizada por RFC9293-STD 7 -TCP.Agosto 2023

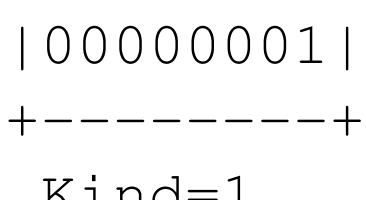
Opciones...

End of Option List

Maximum Segment Size



No-Operation



Opciones...

```
TCP Window Scale option (WSopt):
   Kind: 3
   Length: 3 bytes
          +---------+---------+---------+
          | Kind=3  |Length=3 |shift.cnt|
          +---------+---------+---------+
               1         1         1
```

```
TCP Timestamps option (TSopt):
   Kind: 8
   Length: 10 bytes
+-------+-------+---------------------+---------------------+
|Kind=8 |  10   |   TS Value (TSval)  TS Echo Reply (TSecr)|
+-------+-------+---------------------+---------------------+
    1       1              4                     4
```

.- Se le anteponen dos NOPs para alinerarlo a enteros de 32 bits

Estamos en:

1. Control de Congestión
   - Slow Start – Congestion Avoidance
   - Fast retransmit – Fast Recovery
   - TCP - Cubic
2. Opciones TCP
3. TCP Selective Acknowledge
4. Performance TCP
5. Explicit Congestion Notification en TCP/IP
6. Active Queue Management
7. Nuevo Transporte

SACK

- ✓ Produce la retransmisión selectiva
- ✓ RFC 2018
- ✓ Comprende dos opciones de TCP
  - ✓ “Sack-Permitted Option”
  - ✓ “Sack Option”

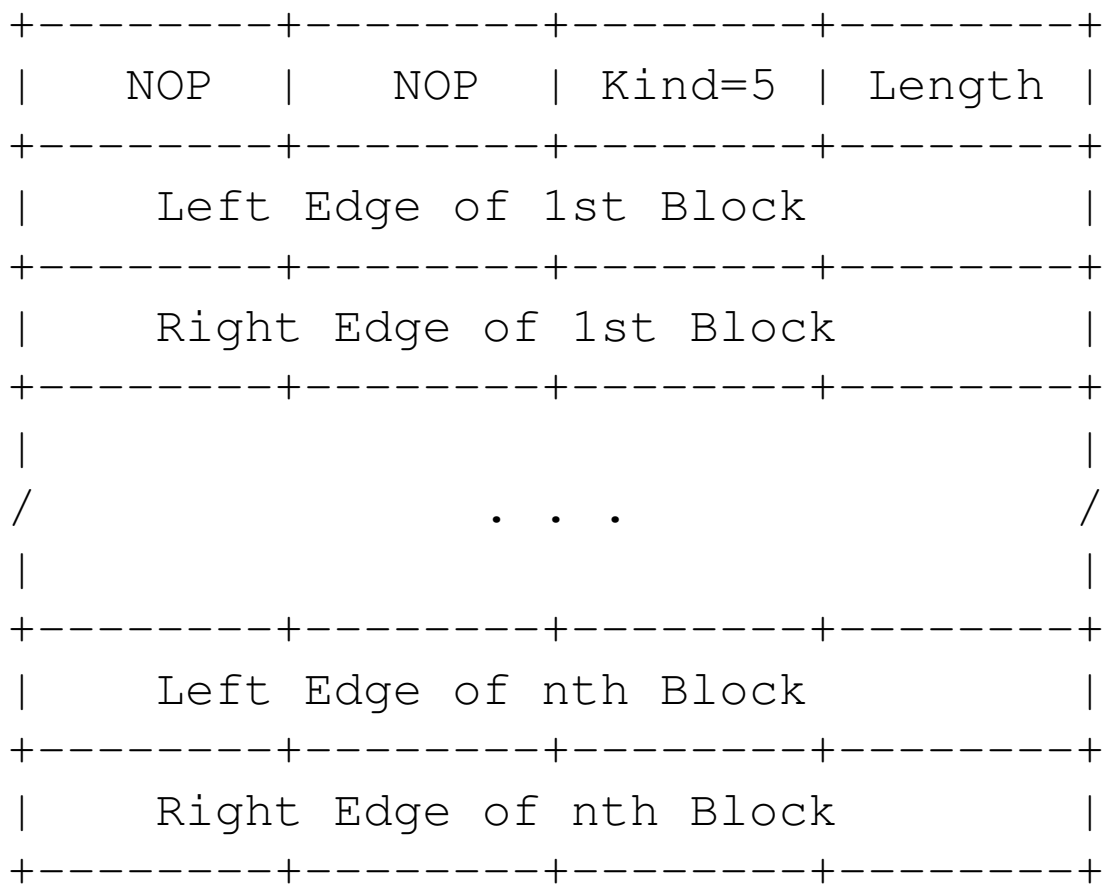
Sack Permitted Option

- ✓ Enviada con el SYN
- ✓ Habilita el uso de SACK

```
+----------+----------+
| Kind=4   | Length=2|
+----------+----------+
```

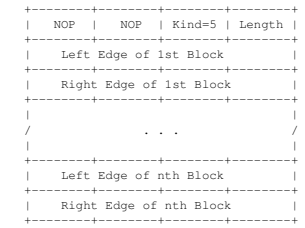
Sack Option – Estructura

Enviada por el receptor



Sack Option – Estructura

Enviada por el receptor



Los NOP se agregan para alinear la opción a palabras de 32 bits (4 bytes).
Left-edge y Right-edge indican el comienzo y fin del bloque de datos correcto que existe en el buffer de recepción de quien envía este segmento
Dado que las opciones tienen una longitud máxima de 40 bytes esto hace que no se puedan indicar más de 4 bloques.

Al recibirse se produce la retransmisión de lo necesario

Ejemplos SACK - rfc 2018

El transmisor envía 8 segmentos de 500 bytes de datos con SN:5000
Caso 1:
Se pierde el primer segmento y llegan OK los 7 restantes:

Triggering Segment	ACK	Left Edge	Right Edge
5000	(lost)		
5500	5000	5500	6000
6000	5000	5500	6500
6500	5000	5500	7000
7000	5000	5500	7500
7500	5000	5500	8000
8000	5000	5500	8500
8500	5000	5500	9000

Ejemplos SACK - rfc 2018...

El transmisor envía 8 segmentos de 500 bytes de datos con SN:5000
Caso 2:
Se pierden los segmentos pares:

Triggering Segment	ACK	1er Bloque		2do Bloque		3er Bloque	
		Left Edge	Right Edge	Left Edge	Right Edge	Left Edge	Right Edge
5000	5500						
5500	(lost)						
6000	5500	6000	6500				
6500	(lost)						
7000	5500	7000	7500	6000	6500		
7500	(lost)						
8000	5500	8000	8500	7000	7500	6000	6500
8500	(lost)						

Ejemplos SACK - rfc 2018...

Continuando con el caso anterior, supongamos que se recibe el cuarto paquete a continuación. Entonces:

Triggering Segment	ACK	1er Bloque		2do Bloque		3er Bloque	
		Left Edge	Right Edge	Left Edge	Right Edge	Left Edge	Right Edge
6500	5500	6000	7500	8000	8500		

Si luego llega el segundo segmento:

Triggering Segment	ACK	1er Bloque		2do Bloque		3er Bloque	
		Left Edge	Right Edge	Left Edge	Right Edge	Left Edge	Right Edge
5500	7500	8000	8500				

Estamos en:

1. Control de Congestión
   - Slow Start – Congestion Avoidance
   - Fast retransmit – Fast Recovery
   - TCP - Cubic
2. Opciones TCP
3. TCP Selective Acknowledge
4. Performance TCP
5. Explicit Congestion Notification en TCP/IP
6. Active Queue Management
7. Nuevo Transporte

TCP Persist Timer

- ✓ Es necesario hacer una suerte de polling
- ✓ Pueden producirse “deadlocks”
- ✓ Al recibirse una ventana de 0 se activa el persist timer
- ✓ Normalmente 5 segundos
- ✓ Cuando expira se envía un segmento de 1 byte para verificar el estado del receptor
- ✓ El receptor le contesta acorde con el estado en que se encuentra
- ✓ Si el receptor vuelve a constestar con ACK , wind=0, entonces el persist timer hace un back-off binario (10, 20, 40 seg...)
- ✓ Al contestar el ACK no validando el byte recibido el emisor continúa enviando este byte de prueba
- ✓ La diferencia con el RTO es que en este caso el emisor envía permanentemente esta prueba hasta que la ventana se incremente o se haga un reset de la conexión

Keepalive Timer

- ✓ En TCP si no hay intercambio de datos no hay tráfico alguno, pero la conexión persiste
- ✓ Hasta que haya una caída en algún extremo o reboot de alguno de los hosts
- ✓ Se define para interrogar al otro extremo por su estado
- ✓ No es parte de la especificación de TCP
- ✓ No se recomienda porque puede provocar caídas en caso que la falla sea transitoria
- ✓ También incrementa el uso del ancho de banda disponible como cualquier otra acción de control
- ✓ En algunos casos es necesario. Ej, Telnet

Keepalive Timer... I

- ✓ En el extremo en que esté habilitado si no hay actividad en 2 horas envía un segmento de prueba
- ✓ El segmento de prueba es similar al de persist timer
- ✓ Puede también enviarse vacío como un ACK pero con un número de secuencia inesperado
- ✓ El receptor se puede encontrar en alguno de estos estados:

  Activo: Responderá al segmento de prueba. El emisor resetea el timer por otras 2 horas. Si en ese intervalo aparece tráfico entonces se vuelve a resetear

  Caído: o en proceso de reboot. El emisor no recibirá respuesta y se genera un timeout a los 75 segundos. Repite 10 veces en intervalos de 75 seg. Si no recibe respuesta finaliza la conexión

Keepalive Timer... II

Fin-reboot: el emisor recibirá una respuesta que será un reset de la conexión

Receptor activo **pero inalcanzable por el emisor:** el emisor no recibe respuesta y genera un timeout de 75 seg al cabo de los cuales retransmite el byte de prueba. Es similar al caso 2

“Wrap around”

- ✓ El número de secuencia tiene 32 bits
- ✓ En velocidades altas el espacio de 32 bits puede reciclar dentro del intervalo de tiempo que un segmento es retardado en la red
- ✓ Para conseguir una operación libre de este error:

$$2^{31}/B > MSL(seg)$$

donde B es la velocidad efectiva del enlace en bytes/seg

$$Twrap = 2^{31}/B$$

Red	B (MBps)	Twrap (seg)
Ethernet	1.25	1700
FDDI	12.5	170
Gigabit	125	17

PAWS

- ✓ Definido para rechazar segmentos duplicados y reencarnaciones
- ✓ Utiliza la opción de timestamp
- ✓ Asume que los segmentos de datos y ACK recibidos contienen un valor de TS monótono no-decreciente
- ✓ Un segmento puede ser descartado como duplicado si se recibe con un valor de TS menor que uno anterior
- ✓ "Menor que" significa que si s y t son valores de TS, entonces

$$s < t \; si \; 0 < (t - s) < 2^{31}$$

- ✓ Los valores de TS enviados en <SYN> y/o <SYN,ACK> inicializan PAWS
- ✓ No requiere sincronización de relojes entre emisor y receptor

Estamos en:

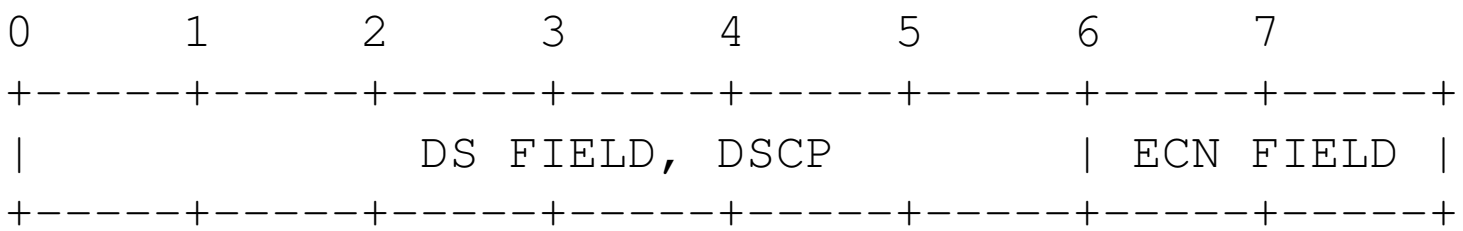
1. Control de Congestión
   - Slow Start – Congestion Avoidance
   - Fast retransmit – Fast Recovery
   - TCP - Cubic
2. Opciones TCP
3. TCP Selective Acknowledge
4. Performance TCP
5. Explicit Congestion Notification en TCP/IP
6. Active Queue Management
7. Nuevo Transporte

Generalidades

- Provee un mecanismo para realizar control de congestión según el modelo centrado en la red.
- Similar a Frame Relay.
- Actúa en IP y se complementa con acciones en TCP.
- En IP procede al marcado de bits
- En TCP las marcas generan el inicio del control de congestión según el mecanismo que tenga implementado

ECN en IP

- ✓ RFC 3168
- ✓ Interactúa con TCP
- ✓ Procede al marcado de bits en el datagrama según criterio prefijado.
- ✓ Utiliza dos bits del campo de TOS del datagrama IP
- ✓ TOS:



Implementación

- Codificación del campo ECN:

ECN FIELD		
0	0	Not-ECT
0	1	ECT(1)
1	0	ECT(0)
1	1	CE

Implementación

- Codificación del campo ECN:

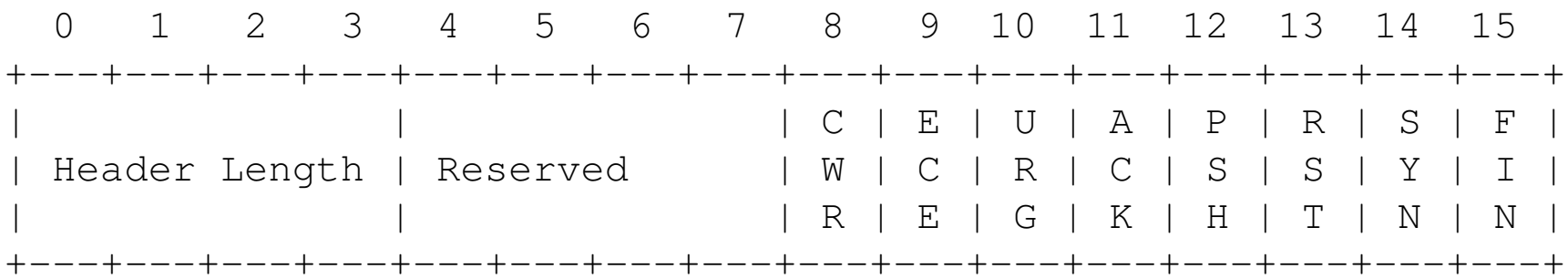
```
+-----+-----+
| ECN FIELD |
+-----+-----+
   0     0       Not-ECT
   0     1       ECT(1)
   1     0       ECT(0)
   1     1       CE
```

Not-ECT indica que el datagrama no acepta trabajar con estos bits
ECT(1) y/o ECT(0) indica que el datagrama está preparado para aceptar el marcado

CE es el caso del datagrama marcado indicando la congestión

ECN en TCP

- Se definen dos nuevos flags, ECE y CWR
- ECE(Explicit Congestion Echo), es enviado por el que recibió el datagrama marcado (CE).
- CWR (Congestion Window Reduced)es seteado y enviado por el que recibió el ECE, acusando recibo del ECE y avisando que ha iniciado control de congestión reduciendo su ventana de congestión.
- La estructura de los bytes 13 y 14 del header de TCP queda:



- Ahora el byte 14 quedó compuesto íntegramente por Flags.

¿Cómo funciona?

1. Se codifica EC(0) o ECT(1) en el datagrama.
2. Un router detecta congestión y al recibir un datagrama ECT lo despacha cambiando los bits a 11 (CE). Todo esto ocurre a nivel IP teniendo en cuenta que es producido por el Router.
3. El que recibe el datagrama con CE avisa a TCP que a la hora de enviar un ACK lo hará con el bit de Flag ECE en 1. El chequeo de CE se hace una vez por ciclo de ventana o RTT que es lo mismo.
4. Al llegar a destino (agente que provoca la congestión), éste realiza dos acciones:
    1. Inicia el mecanismo de control de congestión que tenga definido.
    2. El próximo datagrama que envíe lo hará con el Flag de CWR activado, avisándole al otro extremo que inició el control de congestión.
5. El otro extremo al recibir un segmento con el Flag de CWR en 1 dejará de enviar los ACK con el Flag de ECE activado.

Estamos en:

1. Control de Congestión
   - Slow Start – Congestion Avoidance
   - Fast retransmit – Fast Recovery
   - TCP - Cubic
2. Opciones TCP
3. TCP Selective Acknowledge
4. Performance TCP
5. Explicit Congestion Notification en TCP/IP
6. Active Queue Management
7. Nuevo Transporte

Características

- RFC 2309 – Reemplazada por RFC 7567(2015)
- Modelo de control de congestión basado en la red
- Controlar la longitud de la cola de salida de los datagramas
- Monitorear el estado de la cola y calcular la longitud promedio
- Marcar los datagramas según esa longitud promedio
- Completa el mecanismo ECN.
- Propone diversos algoritmos
- Ejemplo: RED (Random Early Detection)

RED

- Define parámetros de la cola:
  - minth: minimum threshold, umbral mínimo de la longitud de la cola
  - maxth: maximum threshold, umbral máximo de longitud de la cola
  - maxp: probabilidad máxima
- Al llegar un datagrama:
  - Si la longitud promedio es menor que minth encola el datagrama
  - Si la longitud promedio es mayor de maxth lo descarta
  - Si la longitud promedio está entre los dos umbrales lo marca con una probabilidad p_a

RED - Cálculos

- La longitud promedio *avg*

$$avg = (1 - w_q)\, avg + w_q\, q$$

, q es la longitud de la cola y w_q un factor de peso.
Cálculo similar al del RTT.

- Con las probabilidades:

$$p_b = maxp \frac{(avg - minth)}{(maxth - minth)}$$

$$p_a = p_b \frac{1}{(1 - count\, p_b)}$$

count es la cantidad de veces que *avg* está dentro de los dos umbrales.

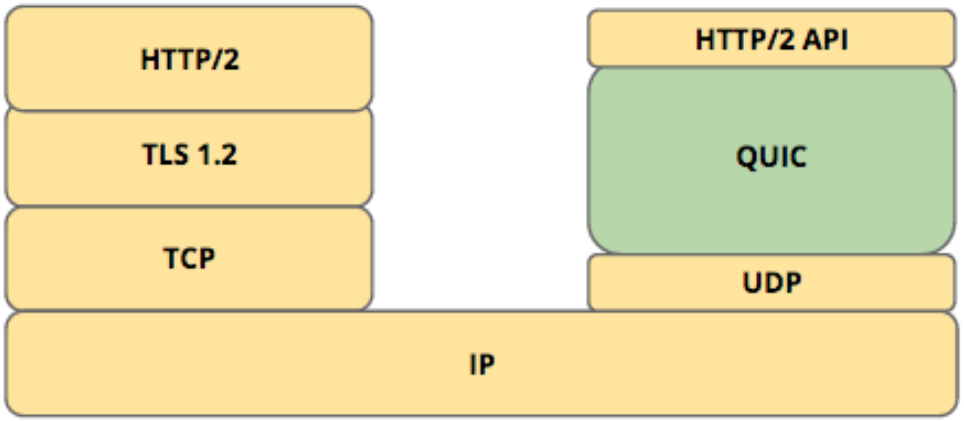
- Si p_a supera a maxp entonces procede a la marca (en el TOS pone los dos bits en 11 o CE

Estamos en:

1. Control de Congestión
   - Slow Start – Congestion Avoidance
   - Fast retransmit – Fast Recovery
   - TCP - Cubic
2. Opciones TCP
3. TCP Selective Acknowledge
4. Performance TCP
5. Explicit Congestion Notification en TCP/IP
6. Active Queue Management
7. Nuevo Transporte

Quic

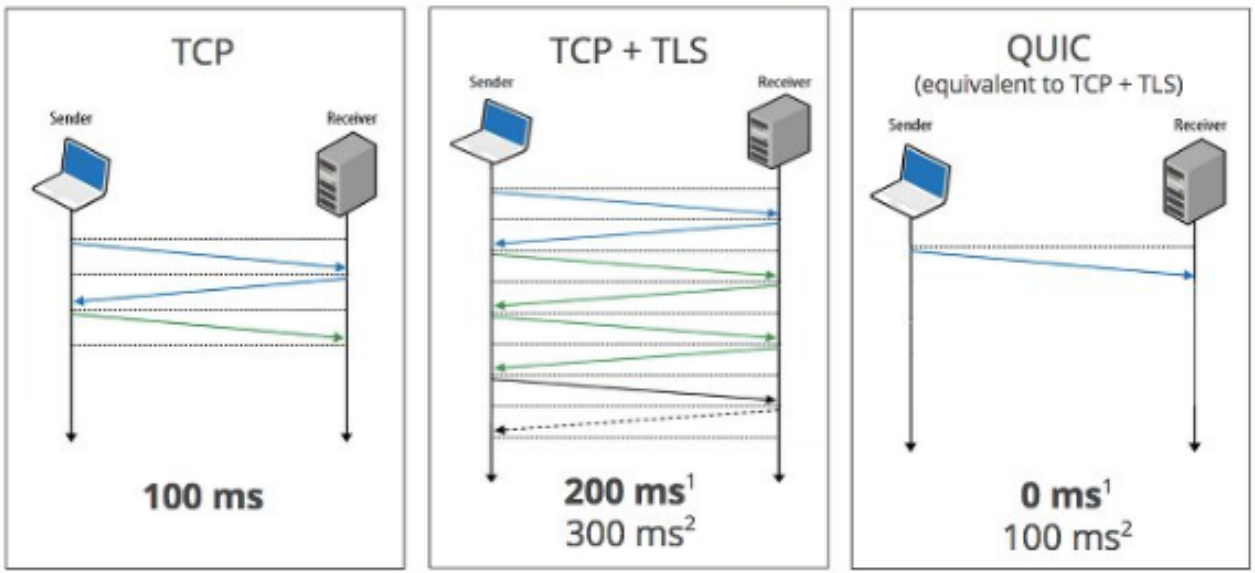
- Desarrollado por Google
- RFC 9000 - QUIC: A UDP-Based Multiplexed and Secure Transport - Mayo 2021
- QUIC Version 2 RFC 9369 . Mayo 2023
- Encapsula a HTTP/3 . RFC 9114 HTTP/3 Junio 2022



Diferencias con TCP

- QUIC está completamente cifrado de forma predeterminada, excepto la versión y algunos bits para determinar el tipo de paquete.
- QUIC retransmite datos con nuevos números de paquetes y los vuelve a cifrar. Esto mejora la detección de pérdidas y la medición de RTT.
- QUIC utiliza el puerto UDP 443 del servidor en lugar de TCP 443 para HTTP/3.
- QUIC no tiene bloqueo de cabecera entre transmisiones confiables.

Quic-Sesión



1 Repite la conexión

2 Primera conexión



Esta obra está sujeta a la licencia Atribución-NoComercial-CompartirIgual 4.0 Internacional (CC BY-NC-SA 4.0) de Creative Commons.

Para detalle de esta licencia visite https://creativecommons.org/licenses/by-nc-sa/4.0/